

CSCI 5540 – Fall 26

Tutorial 5

DENG Yangtao

Credits to CMU Machine Learning Systems

What we will cover

1 Tensor parallelism

Split one layer; track partial results

2 Transformer examples

Pair linear layers and divide attention heads

3 Pipeline schedules

GPipe, 1F1B, and interleaving

4 Putting it together

Combine data, tensor, and pipeline parallelism

Main question: what does each GPU compute, store, and communicate?

From sharding model state to splitting computation

Method	What is divided?	What does a GPU compute?
Plain DP	Batches	All layers, full operators
ZeRO-3 / FSDP	Persistent model state	All layers, after gathering weights
Tensor parallelism (TP)	Tensor dimensions within a layer	A slice of each operator
Pipeline parallelism (PP)	Groups of layers	Only its assigned stages

Model parallelism splits computation: within layers (TP) or across layers (PP).

Tensor parallelism: partition the output features

$$X \in \mathbb{R}^{B \times C_{in}}, \quad W = [W_0 \ W_1], \quad Y = [XW_0 \ XW_1]$$

Same input on both GPUs: $X = [1, 2]$

GPU 0: columns 0–1

$$W_0 = \begin{bmatrix} 1 & 3 \\ 2 & 4 \end{bmatrix}$$

$$Y_0 = [5, 11]$$

GPU 1: columns 2–3

$$W_1 = \begin{bmatrix} 5 & 7 \\ 6 & 8 \end{bmatrix}$$

$$Y_1 = [17, 23]$$

Output is sharded: $Y = [5, 11 \mid 17, 23]$. No reduction is needed in forward.

Tensor parallelism: split inputs, then sum outputs

$$X = [X_0 \ X_1], \quad W = \begin{bmatrix} W_0 \\ W_1 \end{bmatrix}, \quad Y = X_0 W_0 + X_1 W_1$$

GPU 0

$$[1, 2] \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = [1, 2]$$



GPU 1

$$[3, 4] \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} = [11, 7]$$



AllReduce SUM: $[1, 2] + [11, 7] = [12, 9]$

The local vectors are partial sums for the same output features.

Backward tells us where the other reduction goes

Layer partition	Forward layout / communication	Backward layout / communication
Output features	Y_0, Y_1 remain separate	$dX = dX_0 + dX_1 \rightarrow \text{sum}$
Input features	$Y = \text{partial}_0 + \text{partial}_1 \rightarrow \text{sum}$	dX_0, dX_1 remain separate

Why sum the input gradient?

$$dX = dY_0 W_0^T + dY_1 W_1^T$$

Both output shards depend on the same X ; their gradient contributions add.

TP owns distinct weight shards. DP synchronizes replicas of the same shard.

Different layers can prefer different strategies

Convolutional feature extractor



Classifier

Large intermediate activations

Relatively few parameters

Many parameters

Smaller activations

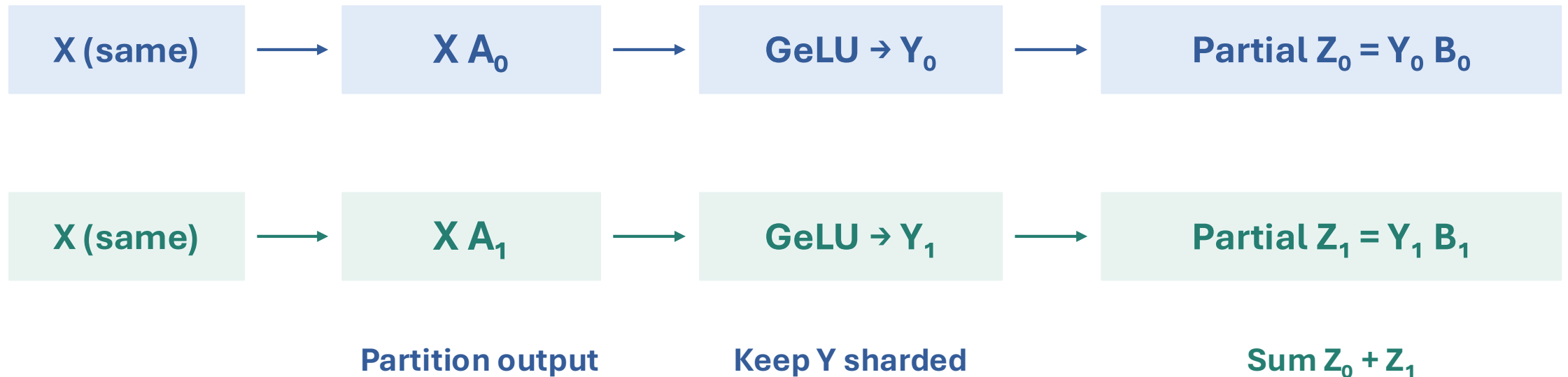
Candidate: DP

Candidate: TP

Choose by tensor shapes and hardware; also count the cost of changing layouts.

Transformer MLP: pair two complementary splits

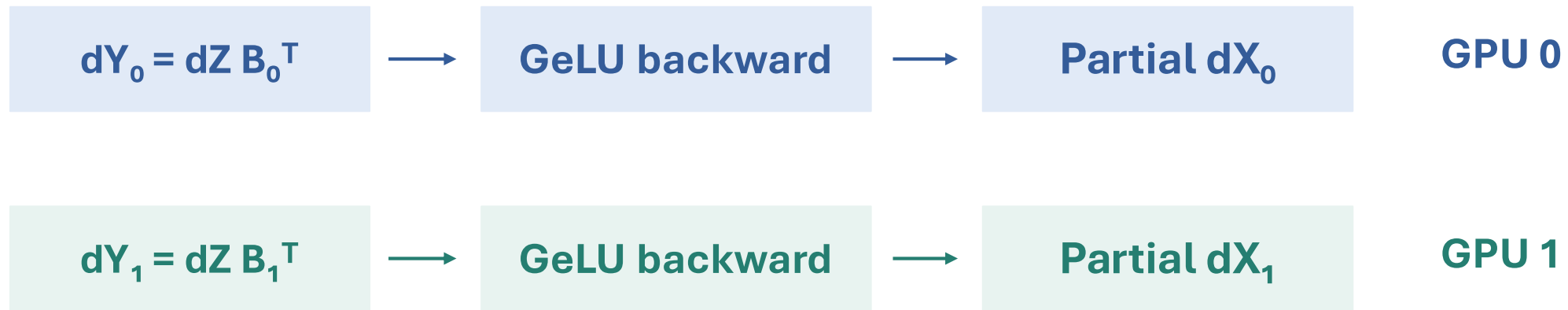
$$Y = \text{GeLU}(XA), \quad Z = YB, \quad A = [A_0 \ A_1], \quad B = \begin{bmatrix} B_0 \\ B_1 \end{bmatrix}$$



No AllGather between the two linear layers; one output AllReduce in forward.

Transformer MLP: follow the backward pass

Both GPUs receive the same dZ



AllReduce SUM at the input: $dX = dX_0 + dX_1$

Basic TP MLP: one AllReduce in forward + one in backward.

Attention: distribute independent heads

Same token representations X

GPU 0

Heads 0, 1, 2, 3

Local Q, K, V $\rightarrow$ attention outputs

Local output-projection shard

GPU 1

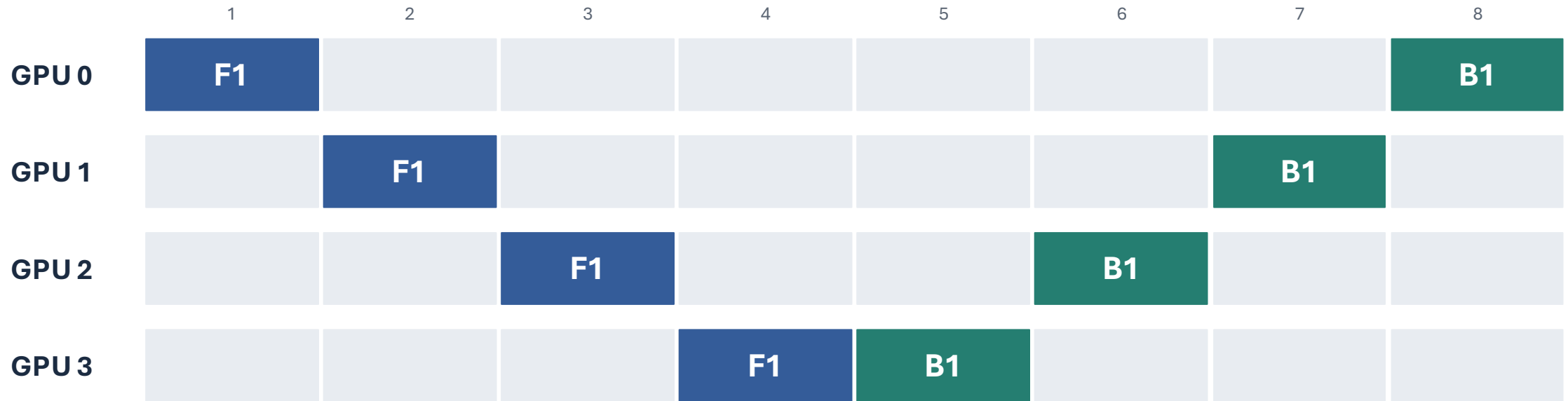
Heads 4, 5, 6, 7

Local Q, K, V $\rightarrow$ attention outputs

Local output-projection shard

Sum the partial output projections; do not gather all heads before projecting.

Layer splitting alone leaves most GPUs idle



Forward

Backward

2 busy slots / 8 total = 25% utilization

Splitting layers solves capacity; micro-batches create pipeline concurrency.

Split a mini-batch into micro-batches

One optimizer step: 16 samples

$\mu 1$: 4 samples

$\mu 2$: 4 samples

$\mu 3$: 4 samples

$\mu 4$: 4 samples

Stage 0



Stage 1



Stage 2



Stage 3

Activations move forward → ← Activation gradients move backward

Same weights for all micro-batches → accumulate gradients → one update.

GPipe example: 4 stages, 4 micro-batches

F1 = forward of μ_1 B1 = backward of μ_1 Gray = idle

	1	2	3	4	5	6	7	8	9	10	11	12	13	14
GPU 0	F1	F2	F3	F4							B4	B3	B2	B1
GPU 1		F1	F2	F3	F4					B4	B3	B2	B1	
GPU 2			F1	F2	F3	F4			B4	B3	B2	B1		
GPU 3				F1	F2	F3	F4	B4	B3	B2	B1			

Fill → forward work → backward work → drain

8 useful slots / 14 total slots = 57.1% utilization (balanced, idealized stages).

Pipeline bubbles: be explicit about the denominator

$$T_{step} = (m + p - 1)(t_f + t_b), \quad T_{useful} = m(t_f + t_b)$$

Quantity	Definition	p = 4, m = 4
Bubble overhead / useful work	$(p - 1) / m$	$3 / 4 = 75\%$
Idle fraction / total time	$(p - 1) / (m + p - 1)$	$3 / 7 = 42.9\%$
Utilization	$m / (m + p - 1)$	$4 / 7 = 57.1\%$

Assumptions: balanced stages, synchronous flush, negligible communication.

Smaller bubbles come with practical tradeoffs

Change	Why it helps	What it costs
More micro-batches, same size	Amortize fill and drain	Larger effective batch
Smaller micro-batches, same batch	More work units to schedule	Smaller, less efficient kernels
Fewer pipeline stages	Less fill / drain overhead	More model state per GPU
Balance stage execution time	Reduce bottleneck stalls	Partitioning / profiling effort

Tune end-to-end step time, not the bubble formula in isolation.

GPipe retains activations until backward can use them

GPU 0: saved activation sets before its first backward



Each set stays live until its matching backward completes.

GPipe peak saved activations: approximately $m \times A$ per stage

A = retained bytes per micro-batch at that stage; checkpointing can reduce A .

1F1B frees activations earlier, with the same bubble

GPipe: fill, then drain

GPU 0 peak: 8 activation sets

GPU 0	F1	F2	F3	F4	F5	F6	F7	F8							B8	B7	B6	B5	B4	B3	B2	B1
GPU 1		F1	F2	F3	F4	F5	F6	F7	F8						B8	B7	B6	B5	B4	B3	B2	B1
GPU 2			F1	F2	F3	F4	F5	F6	F7	F8				B8	B7	B6	B5	B4	B3	B2	B1	
GPU 3				F1	F2	F3	F4	F5	F6	F7	F8	B8	B7	B6	B5	B4	B3	B2	B1			

1F1B: warmup → alternate → drain

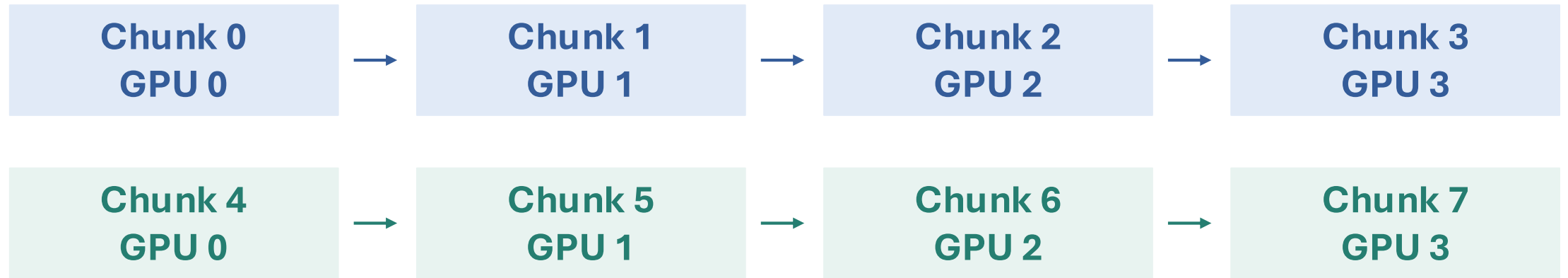
GPU 0 peak: 4 activation sets

GPU 0	F1	F2	F3	F4				B1	F5	B2	F6	B3	F7	B4	F8	B5		B6		B7		B8
GPU 1		F1	F2	F3			B1	F4	B2	F5	B3	F6	B4	F7	B5	F8	B6		B7		B8	
GPU 2			F1	F2		B1	F3	B2	F4	B3	F5	B4	F6	B5	F7	B6	F8	B7		B8		
GPU 3				F1	B1	F2	B2	F3	B3	F4	B4	F5	B5	F6	B6	F7	B7	F8	B8			

$p = 4$, $m = 8$: both take 22 slots. Peak saved sets fall from $O(m)$ to $O(p)$.

Interleaved 1F1B: place several chunks on each GPU

Example: $p = 4$ GPUs, $v = 2$ model chunks per GPU



Model order: chunk 3 $\rightarrow$ chunk 4 returns to GPU 0

$$\text{Bubble overhead} \approx \frac{p-1}{mv} : \quad \frac{3}{8} = 37.5\% \longrightarrow \frac{3}{16} = 18.75\%$$

Shorter chunks reduce ideal bubble time, while more stage boundaries add transfers.

3D parallelism: 8 GPUs = 2 DP × 2 PP × 2 TP

DP replica 0: different examples

PP stage 0

GPU 0 / TP 0

→

PP stage 1

GPU 2 / TP 0

GPU 1 / TP 1

GPU 3 / TP 1

DP replica 1: different examples

PP stage 0

GPU 4 / TP 0

→

PP stage 1

GPU 6 / TP 0

GPU 5 / TP 1

GPU 7 / TP 1

DP sync groups: (0,4), (1,5), (2,6), (3,7)

Total GPUs = $d \times p \times t$. Global batch = $d \times m \times b$ (equal micro-batches).

Choose each dimension for the problem it solves

Dimension	Main benefit	Main limitation
Data parallelism	Scale across independent examples	Full replicas in plain DP; gradient sync
Tensor parallelism	Split large layer tensors / operators	Frequent activation communication
Pipeline parallelism	Split model depth across devices	Bubbles and stage imbalance

Capacity first, then throughput: measure memory, communication, and stage balance.

Three ideas to remember

1

Tensor parallelism

Partition output features, or sum partial outputs.

2

Pipeline parallelism

Micro-batches overlap stages; schedules control bubbles and activation lifetime.

3

Combined parallelism

$DP \times TP \times PP$ distributes examples, operators, and model depth.

Thanks!